

EXPERIMENT-6: XML bookstore with DTD and XSD validation.

Create an XML for the bookstore and validate it using both DTD and XSD.

Objective : Learn XML schema design and validate data structures with document definitions.

Tools : XML, DTD, XSD, validation

Outcome : A bookstore XML document with both DTD and XSD validation support.

Answer:

Define bookstore XML and create matching DTD/XSD schemas to enforce structure and data types.

📁 File Structure

Files

File	Purpose
bookstore.xml	Sample XML data (3 books)
bookstore.dtd	DTD defining structure rules
bookstore.xsd	XSD defining structure + data types
validate.js	Node.js script that runs both validations
package.json	Project metadata / npm script

File Name: bookstore.xml

Code:

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book id="b1">
    <title>Node.js Essentials</title>
    <author>Sarah Mitchell</author>
    <price>299.00</price>
  </book>
  <book id="b2">
    <title>Mastering XML Schemas</title>
    <author>Rajesh Kumar</author>
```

```
<price>450.50</price>
</book>
<book id="b3">
  <title>Web Development with Express</title>
  <author>Emily Chen</author>
  <price>375.00</price>
</book>
</bookstore>
```

File Name: bookstore.dtd

```
<!-- bookstore.dtd -->
<!ELEMENT bookstore (book+)>

<!ELEMENT book (title, author, price)>
<!ATTLIST book id ID #REQUIRED>

<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

File Name: bookstore.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="bookstore">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="author" type="xs:string"/>
              <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```

        </xs:sequence>
        <xs:attribute name="id" type="xs:ID" use="required"/>
    </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:element>

</xs:schema>

```

File Name: validate.js

```

const { execFile } = require("child_process");
const fs = require("fs");
const os = require("os");
const path = require("path");

const XML_FILE = path.join(__dirname, "bookstore.xml");
const DTD_FILE = path.join(__dirname, "bookstore.dtd");
const XSD_FILE = path.join(__dirname, "bookstore.xsd");

function runCommand(command, args, options = {}) {
    return new Promise((resolve) => {
        execFile(command, args, { windowsHide: true, ...options }, (error, stdout, stderr) => {
            resolve({
                success: !error,
                code: error && error.code ? error.code : 0,
                message: (stderr || stdout || "").trim(),
            });
        });
    });
}

```

```

async function isCommandAvailable(command) {
  const lookupCommand = process.platform === "win32" ? "where" : "which";
  const result = await runCommand(lookupCommand, [command]);
  return result.success;
}

function runXmllint(args) {
  return runCommand("xmllint", args);
}

function createDtdTempXml() {
  const tempDir = fs.mkdtempSync(path.join(os.tmpdir(), "bookstore-xml-"));
  const dtdTempPath = path.join(tempDir, "bookstore.dtd");
  fs.copyFileSync(DTD_FILE, dtdTempPath);

  const xml = fs.readFileSync(XML_FILE, "utf8");
  const withDoctype = xml.replace(
    /^(<?\?xml[^\?>]*\?>\s*)?/,
    "$1<!DOCTYPE bookstore SYSTEM \"bookstore.dtd\">\n"
  );

  const tempXmlPath = path.join(tempDir, "bookstore-dtd.xml");
  fs.writeFileSync(tempXmlPath, withDoctype, "utf8");
  return { tempDir, tempXmlPath };
}

async function runJavaValidation(mode, xmlPath, schemaPath) {
  const compileCheck = await runCommand("javac", ["-version"]);
  if (!compileCheck.success) {
    return {
      success: false,
      message:
        "Java is not installed or 'javac' is not on PATH. Install JDK 17+ or add xmllint to your environment.",
    };
  }
}

```

```
};  
}
```

```
const tempDir = fs.mkdtempSync(path.join(os.tmpdir(), "xml-java-validator-"));
const helperFile = path.join(tempDir, "XmlValidator.java");
const helperSource = `
import java.io.File;
import java.io.FileInputStream;
import javax.xml.XMLConstants;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.stream.StreamSource;
import javax.xml.validation.Schema;
import javax.xml.validation.SchemaFactory;
import javax.xml.validation.Validator;
import org.xml.sax.InputSource;

public class XmlValidator {
    private static void validateDtd(String xmlPath, String dtdPath) throws Exception {
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        factory.setNamespaceAware(true);
        factory.setValidating(true);
        factory.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
        factory.setFeature("http://apache.org/xml/features/nonvalidating/load-dtd-grammar",
true);
        factory.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd",
true);

        DocumentBuilder builder = factory.newDocumentBuilder();
        builder.setEntityResolver((publicId, systemId) -> {
            if (systemId != null && systemId.endsWith("bookstore.dtd")) {
                return new InputSource(new FileInputStream(dtdPath));
            }
            return null;
        });
    }
}
```

```
});
```

```
builder.parse(new File(xmlPath));  
}
```

```
private static void validateXsd(String xmlPath, String xsdPath) throws Exception {  
    SchemaFactory schemaFactory =  
SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);  
    Schema schema = schemaFactory.newSchema(new File(xsdPath));  
    Validator validator = schema.newValidator();  
    validator.validate(new StreamSource(new File(xmlPath)));  
}
```

```
public static void main(String[] args) {  
    try {  
        if (args.length < 3) {  
            throw new IllegalArgumentException("Missing validation arguments.");  
        }  
  
        String mode = args[0];  
        String xmlPath = args[1];  
        String schemaPath = args[2];  
  
        if ("DTD".equals(mode)) {  
            validateDtd(xmlPath, schemaPath);  
        } else if ("XSD".equals(mode)) {  
            validateXsd(xmlPath, schemaPath);  
        } else {  
            throw new IllegalArgumentException("Unknown validation mode: " + mode);  
        }  
  
        System.out.println("VALID");  
    } catch (Exception ex) {  
        System.out.println("INVALID");  
    }  
}
```

```

    System.out.println(ex.getMessage());
    ex.printStackTrace(System.out);
    System.exit(1);
  }
}
}
`;

```

```

fs.writeFileSync(helperFile, helperSource, "utf8");

```

```

const compileResult = await runCommand("javac", [helperFile], { cwd: tempDir });
if (!compileResult.success) {
  return {
    success: false,
    message: compileResult.message || "Java compilation failed.",
  };
}

```

```

const runResult = await runCommand("java", ["-cp", tempDir, "XmlValidator", mode,
xmlPath, schemaPath], {
  cwd: tempDir,
});

```

```

if (runResult.success) {
  return {
    success: true,
    message: "VALID",
  };
}

```

```

return {
  success: false,
  message: runResult.message || "Validation failed using the Java fallback.",
};

```

```
}
```

```
async function validateDtd() {  
  if (await isCommandAvailable("xmllint")) {  
    return runXmllint(["--noout", "--dtdvalid", DTD_FILE, XML_FILE]);  
  }  
}
```

```
const { tempXmlPath } = createDtdTempXml();  
return runJavaValidation("DTD", tempXmlPath, DTD_FILE);  
}
```

```
async function validateXsd() {  
  if (await isCommandAvailable("xmllint")) {  
    return runXmllint(["--noout", "--schema", XSD_FILE, XML_FILE]);  
  }  
}
```

```
return runJavaValidation("XSD", XML_FILE, XSD_FILE);  
}
```

```
function printResult(label, result) {  
  const line = "-".repeat(50);  
  console.log(line);  
  console.log(`${label}`);  
  console.log(line);  
  if (result.success) {  
    console.log("✅ VALID: XML conforms to the schema.\n");  
  } else {  
    console.log("❌ INVALID: Validation errors found:");  
    console.log(result.message + "\n");  
  }  
}
```

```
async function main() {
```



```
console.log("\n=== XML Bookstore Validation (DTD + XSD) ===\n");
```

```
const dtdResult = await validateDtd();
```

```
printResult("DTD Validation (bookstore.dtd)", dtdResult);
```

```
const xsdResult = await validateXsd();
```

```
printResult("XSD Validation (bookstore.xsd)", xsdResult);
```

```
console.log("=".repeat(50));
```

```
console.log("SUMMARY");
```

```
console.log("=".repeat(50));
```

```
console.log(`DTD: ${dtdResult.success ? "PASS" : "FAIL"}`);
```

```
console.log(`XSD: ${xsdResult.success ? "PASS" : "FAIL"}`);
```

```
console.log("");
```

```
if (!dtdResult.success || !xsdResult.success) {
```

```
    process.exitCode = 1;
```

```
}
```

```
}
```

```
main();
```

File Name: package.json

```
{
```

```
  "name": "bookstore-xml-validation",
```

```
  "version": "1.0.0",
```

```
  "description": "Week-6 Lab: XML bookstore validated using both DTD and XSD in Node.js",
```

```
  "main": "validate.js",
```

```
  "scripts": {
```

```
    "validate": "node validate.js"
```

```
  },
```

```
  "license": "MIT"
```

```
}
```

